CS214 – Data Structure Assignment 2

Introduction

This assignment consists of two problems. For each problem, you are required to carefully read the description and submit all required C++ files along with a report.

In the first problem, you will apply **Binary Search Trees (BST)** and **AVL Trees** to build a smart library book management system. You must implement both data structures from scratch.

In the second problem, you will apply the **Heap** data structure to build an emergency room priority system. You are required to implement the heap manually without using any built-in STL structures such as priority_queue.

For each problem, you must submit:

- The complete C++ implementation
- A sample run demonstrating the functionality (included in the main function)

Problem 1: Smart Library Book Management System

Requirements

1. Book Structure

Each book contains:

```
struct Book {
   int id;
   string title;
   string author;
};
```

Part 1: BST Implementation

Implement a BST with:

- Insert a book
- Delete a book
- Search by ID
- In-order traversal (display sorted books)

Part 2: AVL Tree Implementation

Implement the same features using AVL Tree:

- Insert (with rotations)
- Delete (with balancing)
- Search
- In-order traversal

You Must include:

- Left rotation
- Right rotation
- Left-Right
- Right-Left

Part 3: Functional Features

Add these real-life operations:

1. Find books in ID range

Example: show all books with IDs between 100 and 200

2. Find closest book ID

If user searches for ID that doesn't exist, return closest match

---

Part 4: Comparison

You must:

- Insert **at least 20–30 books**
- Use two cases:
    1. Random IDs
    2. Sorted IDs

Then compare:

- Height of BST vs AVL
- Search steps (or time)

Deliverables for problem 1:

1. C++ Code (BST + AVL)
2. Sample input/output
3. Short report (1–2 pages):
    - Differences observed
    - When to use BST vs AVL

Problem 2: Emergency Room Priority System (Heap)

Scenario (Real-Life)

A hospital emergency room receives patients continuously.

Each patient has:

- **ID**
- **Name**
- **Severity Level (1–10)** → higher = more critical
- **Arrival Time**

Patients must be treated based on:

1. Higher severity first
2. If equal → earlier arrival first

Data Structure Requirement

You must implement:

vector<Patient> heap;

Heap priority:

(severity DESC, arrivalTime ASC)

Patient Structure

```
struct Patient {
   int id;
   string name;
   int severity;
   int arrivalTime;
};
```

Required Operations

1. Insert Patient

- Add new patient
- Maintain heap property (heapify up)

---

2. Treat Next Patient

- Remove highest priority patient
- Heapify down

---

3. View Next Patient

- Peek top without removing

---

4. Update Severity

- Change severity of a patient
- Reheapify accordingly

---

5. Display All Patients (Level Order)

Constraints (important)

- NOT allowed to use STL priority_queue in part 1

Part 2: Analysis

You must:

- Insert **20+ patients**
- Compare:
    - Manual heap vs STL priority_queue

- Time complexity
- Ease of use

Deliverables

1. C++ code (heap implementation)
2. Sample run
3. Report:
    - How heap works
    - Why it's efficient
    - Comparison with other structures
